UNIVERSITY OF PASSAU

Faculty of Computer Science and Mathematics

Reverse Engineering – Decompilation

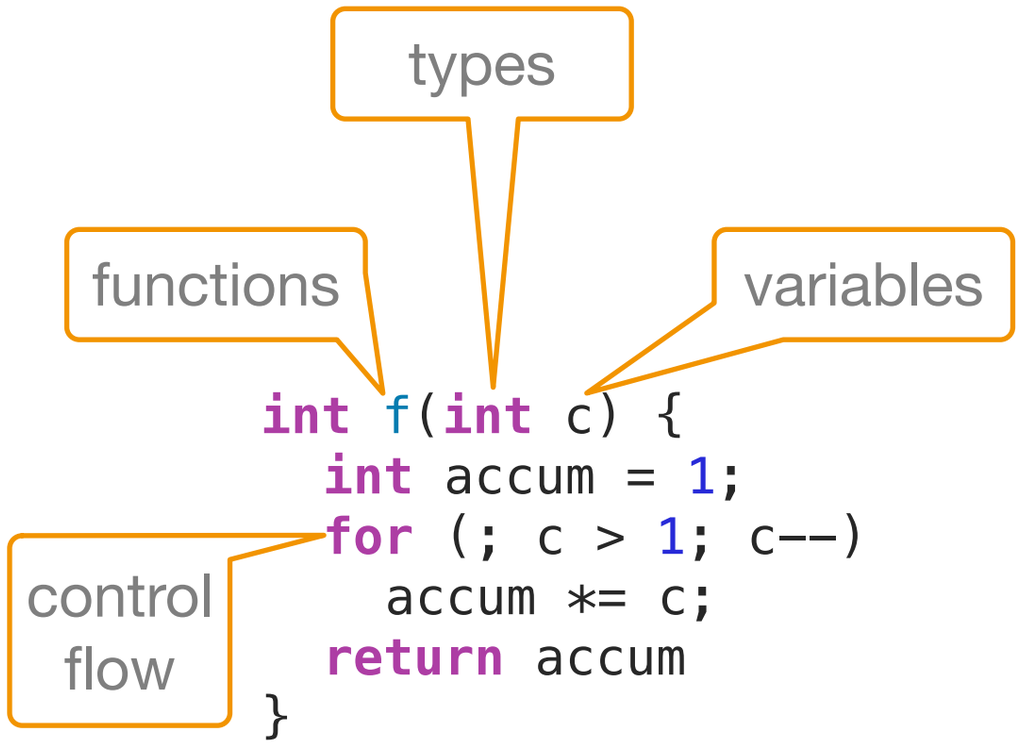


Prof. Dr.-Ing. Christian Hammer

WS 2026/27

Which one would you prefer?

- push %ebp
- mov %esp,%ebp
- sub $0x10,%esp
- movl $0x1,-0x4 (%ebp)
- jmp 1d <f+0x1d>
- mov -0x4(%ebp) , %eax
- imul 0x8 (%ebp),%eax
- mov %eax,-0x4(%ebp)
- subl $0x1, 0x8(%ebp)
- cmpl $0x1, 0x8(%ebp)
- jg f <f+0xf>
- mov -0x4(%ebp), %eax
- leave
- ret



Compiler vs. Decompiler

- Compiler translates source code to assembly
- Decompiler converts assembly code into source code ‘high level’ language (usually C)
- Compilation loses comments, file structure, etc.
- Untyped machine language: is it an integer, a pointer, or something else?
- Compiler optimizations change code structure: function inlining, removal of loops, code duplication (loop unrolling), etc.

- Statements

x := *E*	m[*E*] := *E*	push *E*	call *E*
if (*E*) goto *E*’	goto *E*	pop *E*	ret

- Expressions

mov eax, 0x401001	eax := 0x401001
cmp eax, ebx	CF := eax $<_u$ ebx OF := eax >= ebx & ((eax < 0 & ebx >= 0) | (eax < 0 & ebx >= 0)) SF := eax < ebx ZF := eax = ebx
jle 0x40100D	if (SF xor OF) | ZF) goto 0x40100D
jmp [ebx+ecx*4]	goto m_{32}[ebx+ecx*4]

IR Translation Example

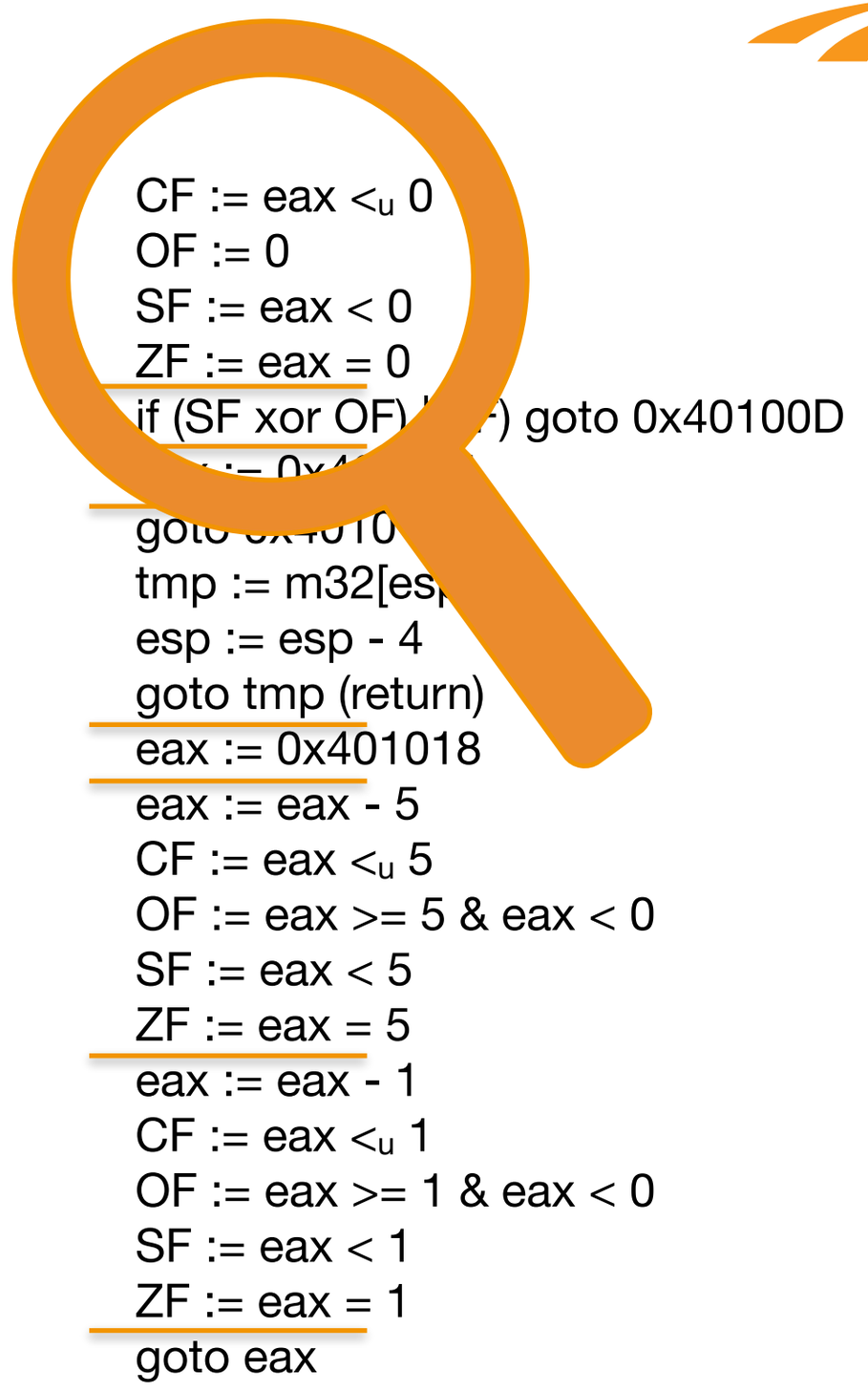
```
00401000 cmp   eax, 0
00401003 jle   0x40100D
00401005 mov   eax, 0x 401001
0040100A jmp   0x401015
0040100C retn
0040100D mov   eax, 0x401018
00401012 sub   eax, 5
00401015 sub   eax, 1
00401018 jmp   eax
```

```
        CF := eax <u 0
        OF := 0
        SF := eax < 0
        ZF := eax = 0
        if (SF xor OF) | ZF) goto 0x40100D
        eax := 0x401001
        goto 0x401015
        tmp := m32[esp]
        esp := esp - 4
        goto tmp (return)
0x40100D: eax := 0x401018
        eax := eax - 5
        CF := eax <u 5
        OF := eax >= 5 & eax < 0
        SF := eax < 5
        ZF := eax = 5
0x401015: eax := eax - 1
        CF := eax <u 1
        OF := eax >= 1 & eax < 0
        SF := eax < 1
        ZF := eax = 1
        goto eax
```

IR Translation Example

Compiler Optimizations:
Constant Propagation

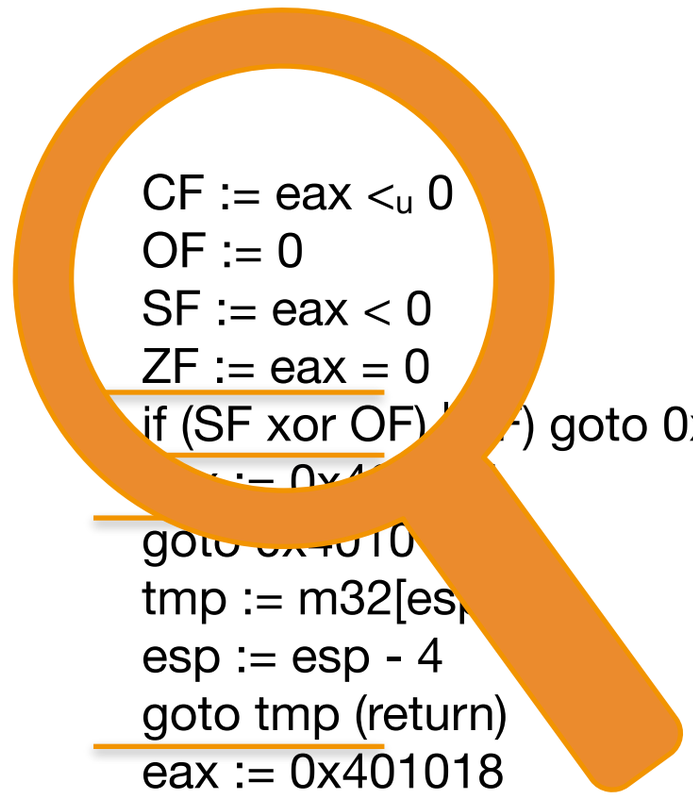
CF := eax <u 0
OF := 0
SF := eax < 0
ZF := eax = 0
if (SF xor OF) | ZF) goto 0x40100D



IR Translation Example

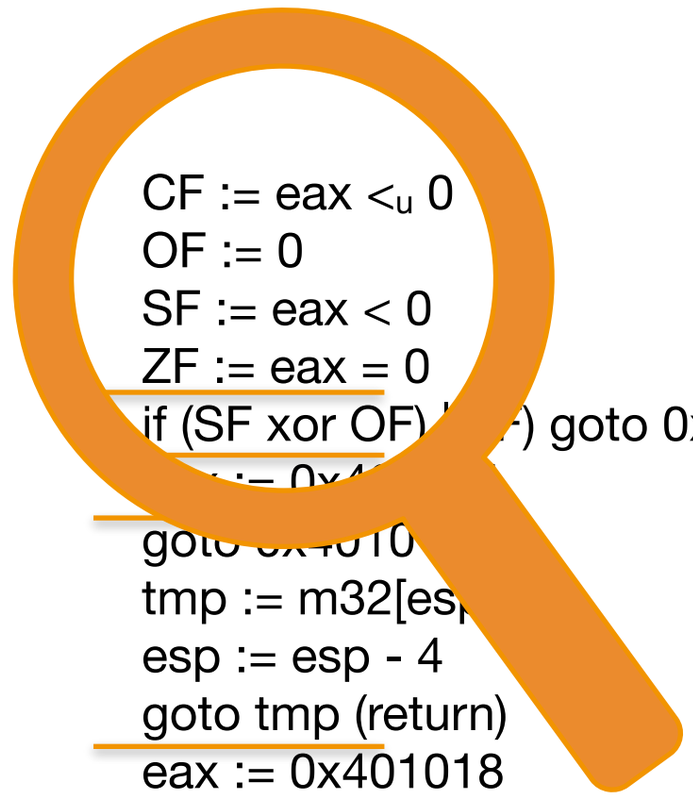
Compiler Optimizations:
Constant Propagation

CF := eax <u 0
OF := 0
SF := eax < 0
ZF := eax = 0
if (SF xor 0) | ZF) goto 0x40100D



Compiler Optimizations:
Constant Propagation

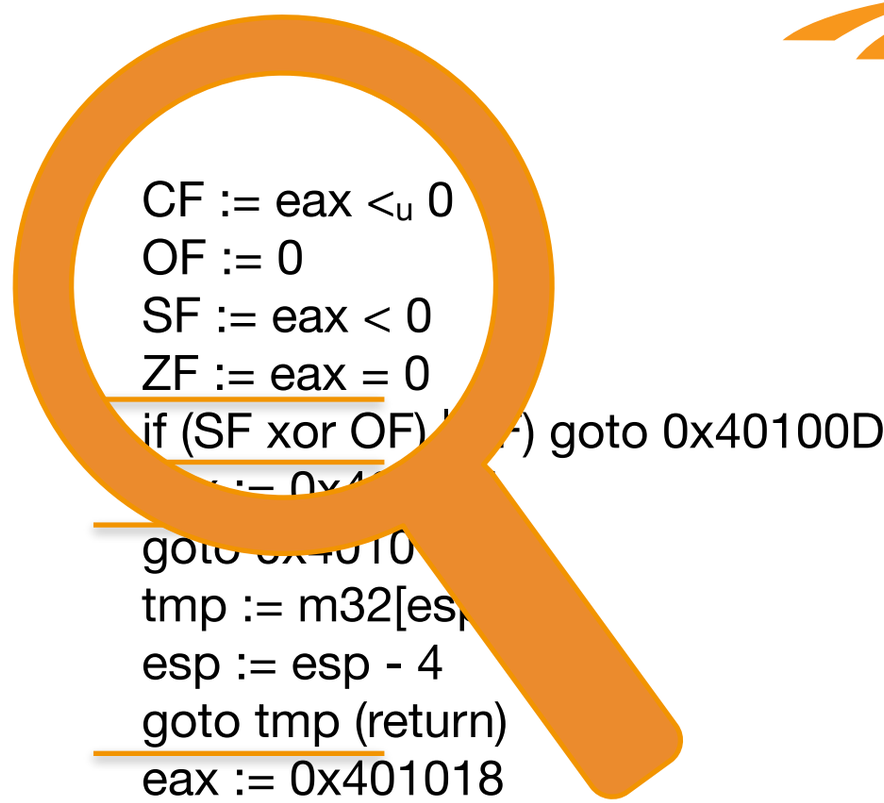
CF := eax <u 0
OF := 0
SF := eax < 0
ZF := eax = 0
if (eax < 0 | eax = 0) goto 0x40100D



IR Translation Example

Compiler Optimizations:
Simplification

CF := eax $<_u$ 0
OF := 0
SF := eax < 0
ZF := eax = 0
if (eax <= 0) goto 0x40100D



Compiler Optimizations:
Dead Code Elimination

if (eax <= 0) goto 0x40100D

```
CF := eax <u 0
OF := 0
SF := eax < 0
ZF := eax = 0
if (SF xor OF) ...F) goto 0x40100D
... := 0x4...
goto 0x4010
tmp := m32[es...
esp := esp - 4
goto tmp (return)
eax := 0x401018
eax := eax - 5
CF := eax <u 5
OF := eax >= 5 & eax < 0
SF := eax < 5
ZF := eax = 5
eax := eax - 1
CF := eax <u 1
OF := eax >= 1 & eax < 0
SF := eax < 1
ZF := eax = 1
goto eax
```

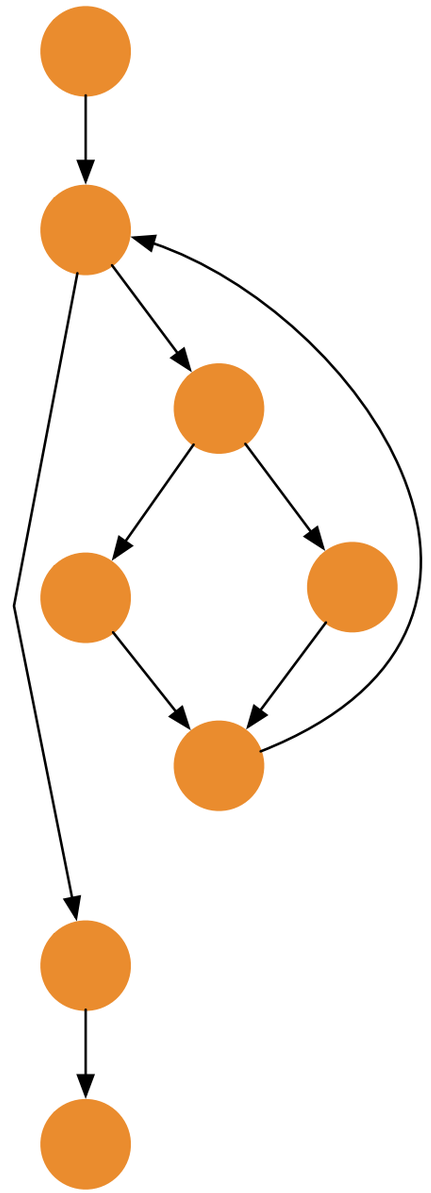
Unstructured

```
    s1;
L1: if (e1) goto L2; else goto L4;
L2: if (e2) goto L4;
L3: s2; goto L1;
L4: s3;
```

Structured

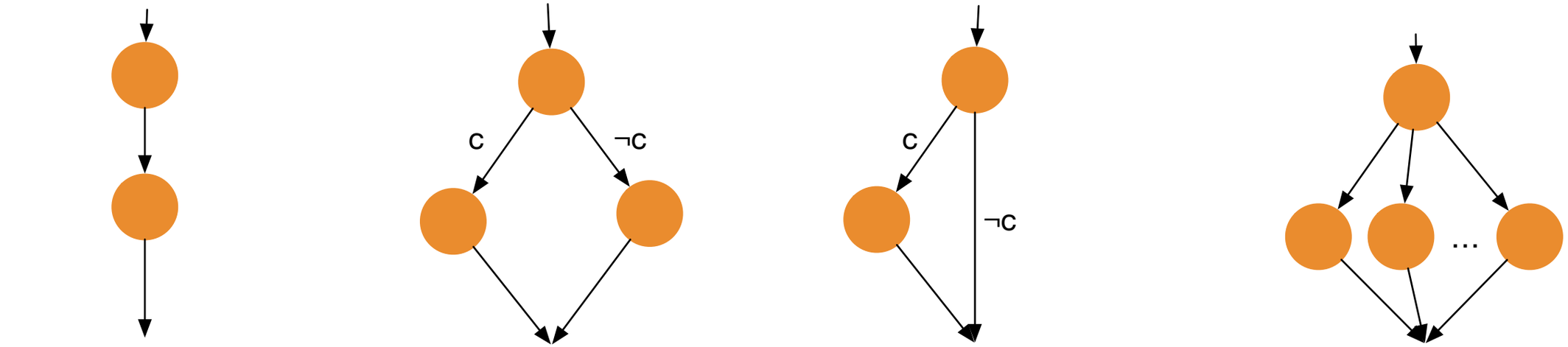
```
s1;
while (e1) {
  if (e2) break;
  s2;
}
s3;
```

- CFG Vertex represents straight-line code
- Edges represent possible control flow transfer
- Challenge: where does `jmp %eax` go?
- Potential solution: Value Set Analysis [Balakrishnan10]



- CF recovery algorithms used in decompilers:
  - Interval analysis [Allen70]
    Identifies intervals or regions
  - Structural analysis [Sharir80]
    Classifies regions into more specific types

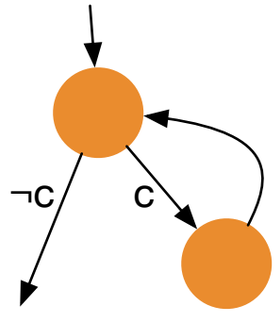
- Iteratively match patterns to CFG, traversing in post-order then collapse region



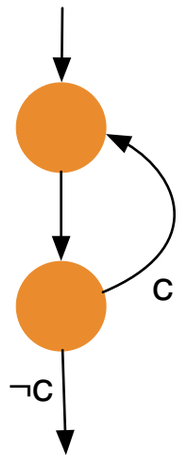
sequence | if-then-else | if-then | switch (compl. or incomplete)

-

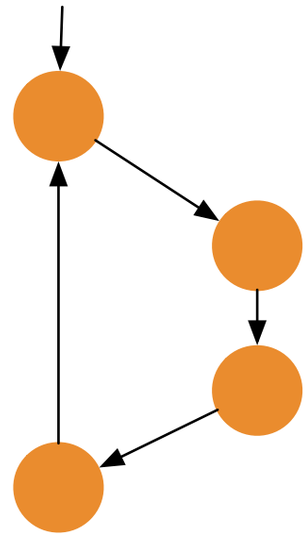
- Iteratively match patterns to CFG, traversing in post-order then collapse region



while



do-while

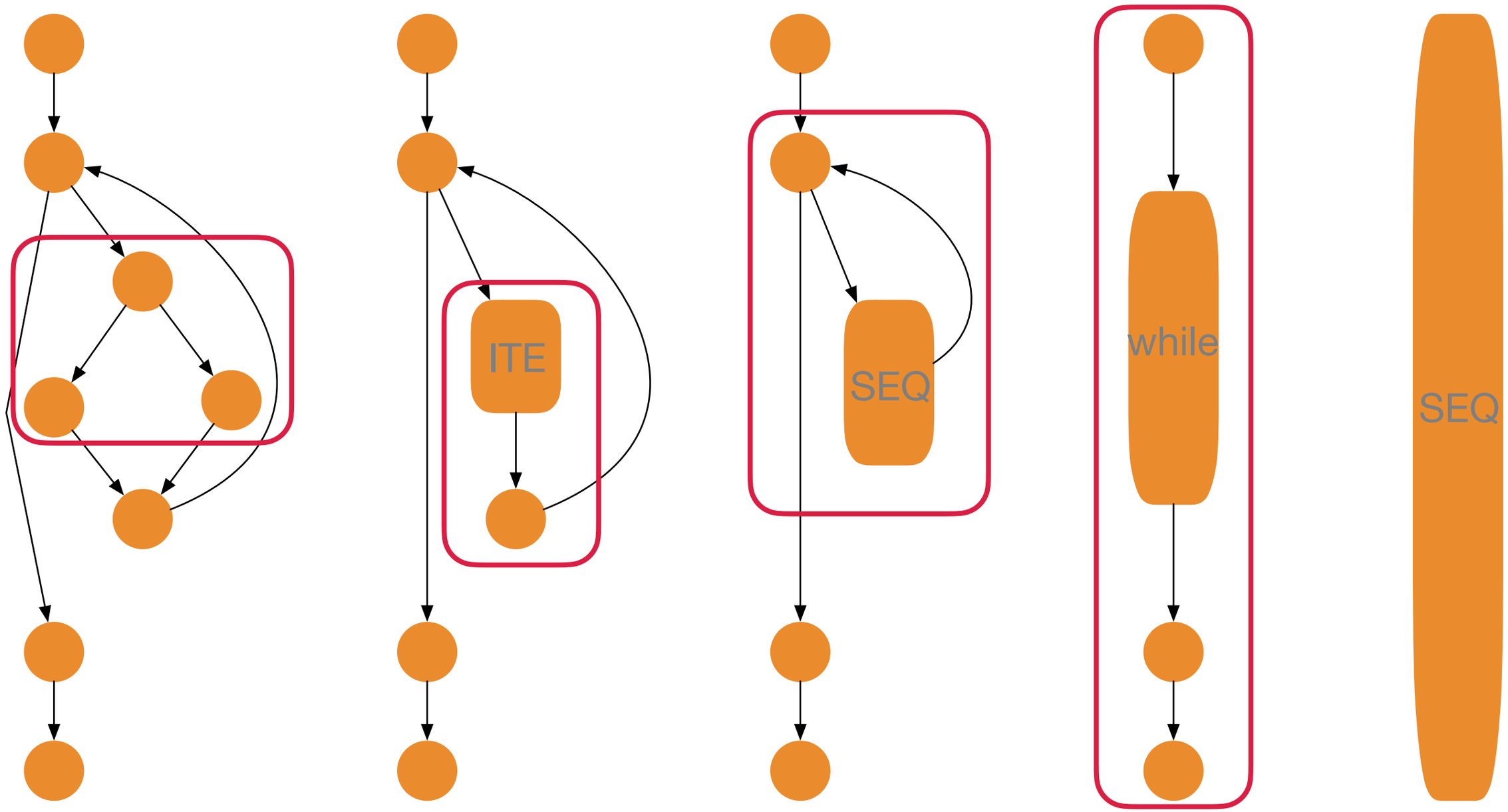


Natural loop
(no exit edges)

- Returns a skeleton: while (e) { if (e') {…} }

Structural Analysis Example

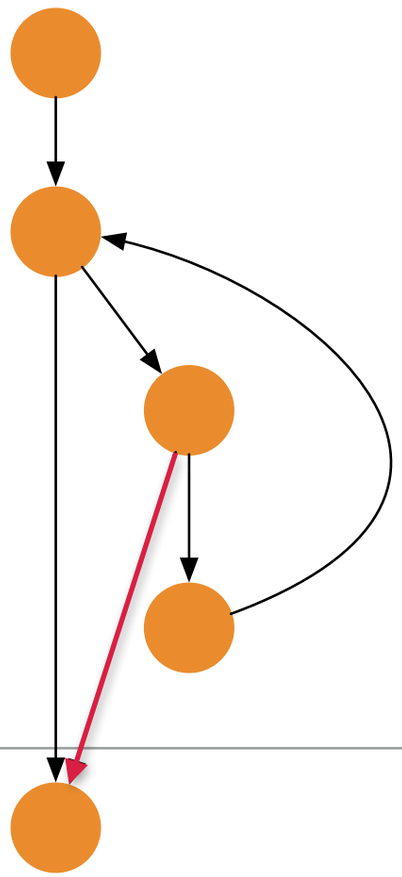
- Skeleton: …; while (…) { if (…) {…} else {…} }; …; …;



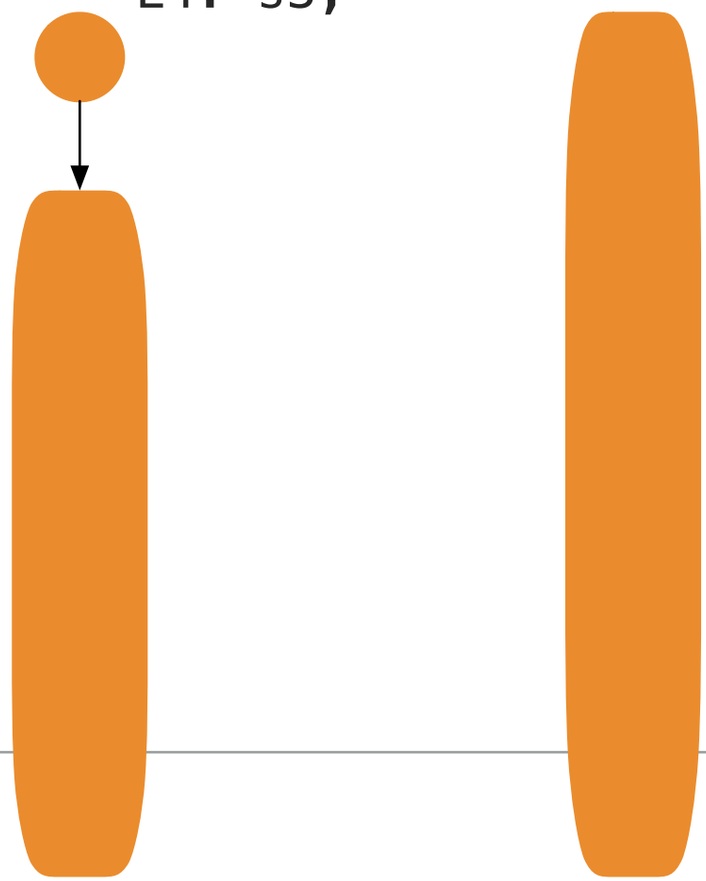
Issues with Structural Analysis

- Fails for unstructured programs
  - break, continue or goto statements
  - Cascades to large subgraphs

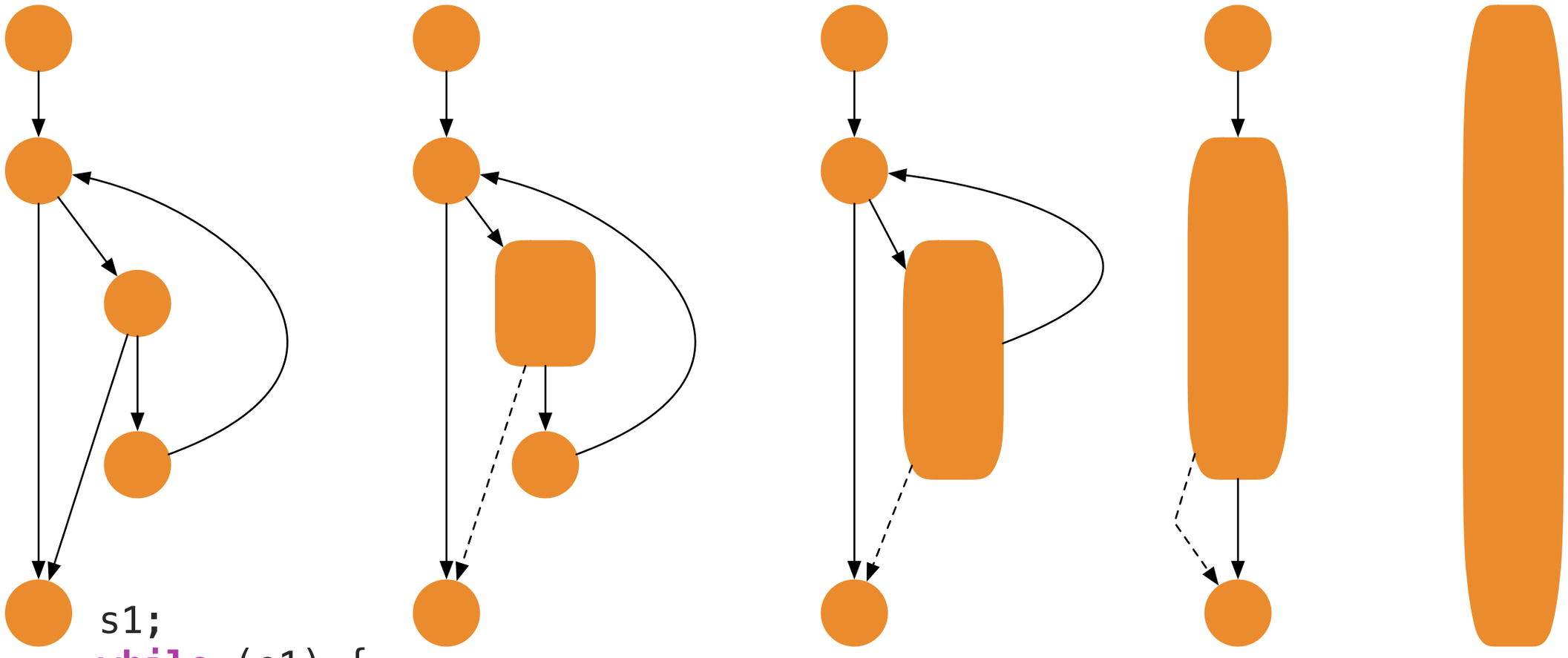
```
s1;
while (e1) {
  if (e2) break;
  s2;
}
s3;
```



```
    s1;
L1: if (e1) goto L2; else goto L4;
L2: if (e2) goto L4;
L3: s2; goto L1;
L4: s3;
```

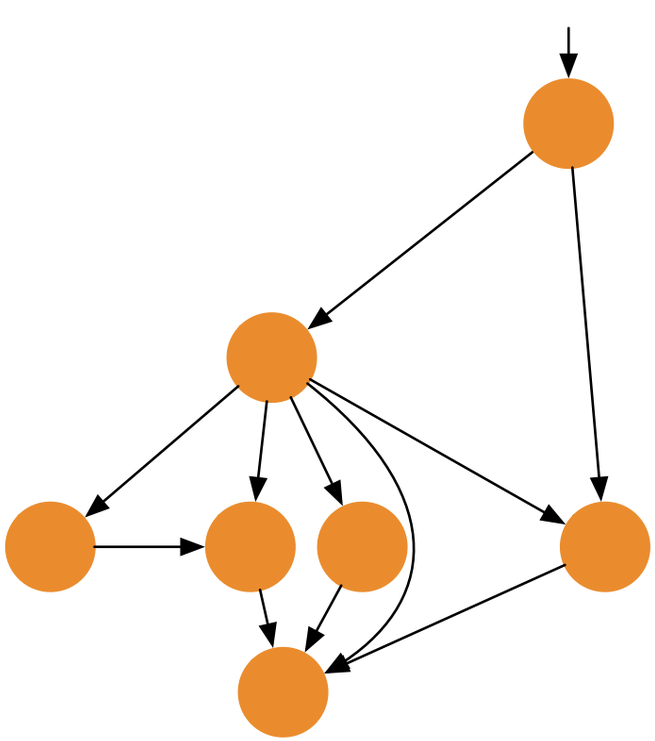


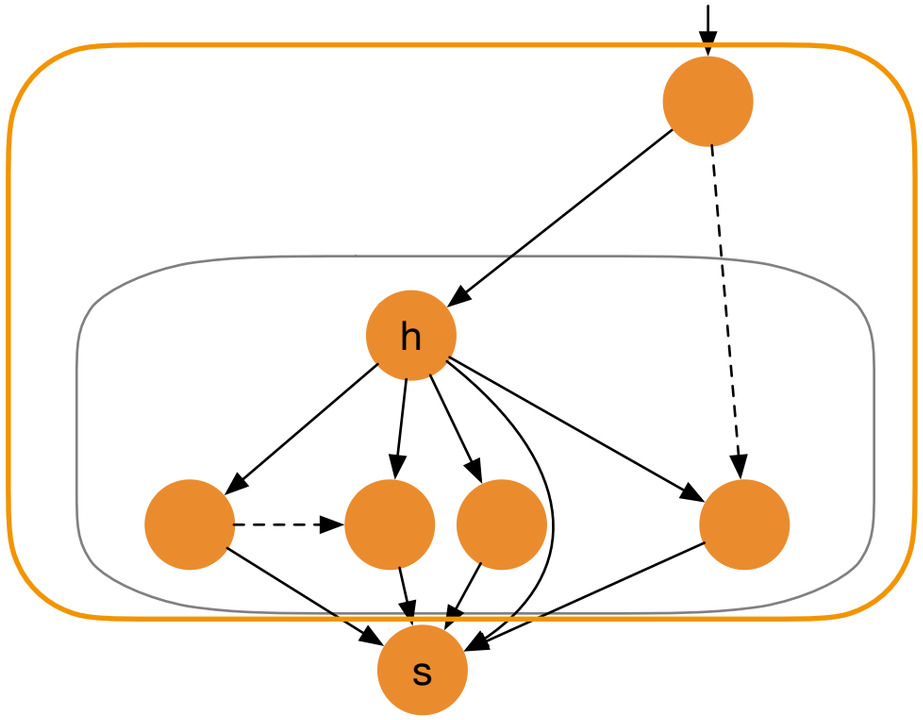
- Remove edges preventing a match
  (will be represented as break, continue, or goto in decompiled code)
- Allows structuring algorithm to make more progress



```
s1;
while (e1) {
  if (e2) break;
  s2;
}
s3;
```

Switch Candidates

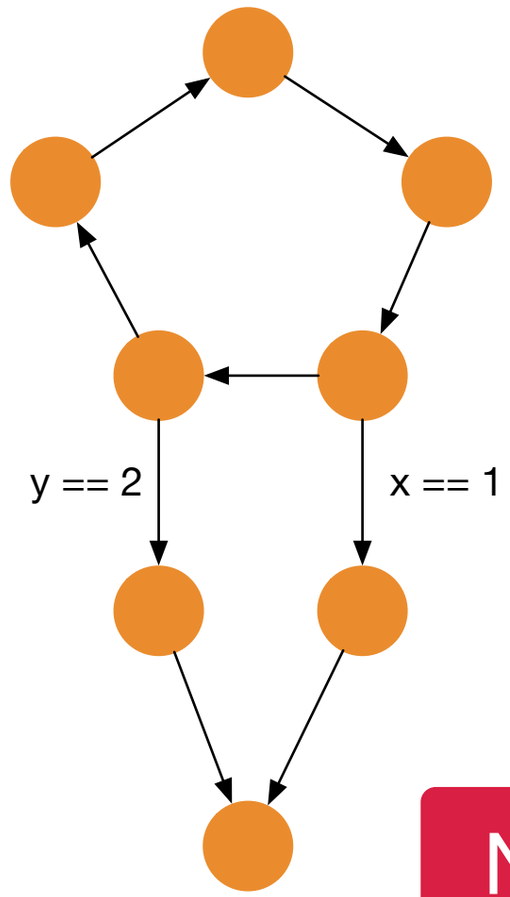


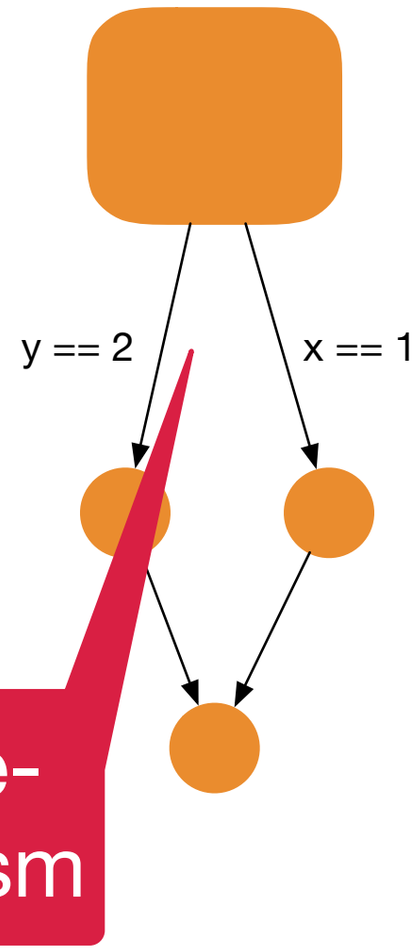


Issues with Structural Analysis (2)

- May lead to incorrect structures for decompilation
  - Not originally intended for this purpose

```
while (true) {
  s1; if (x == 1) goto L2;
  if (y == 2) goto L1;
}
L2:
L1:
```





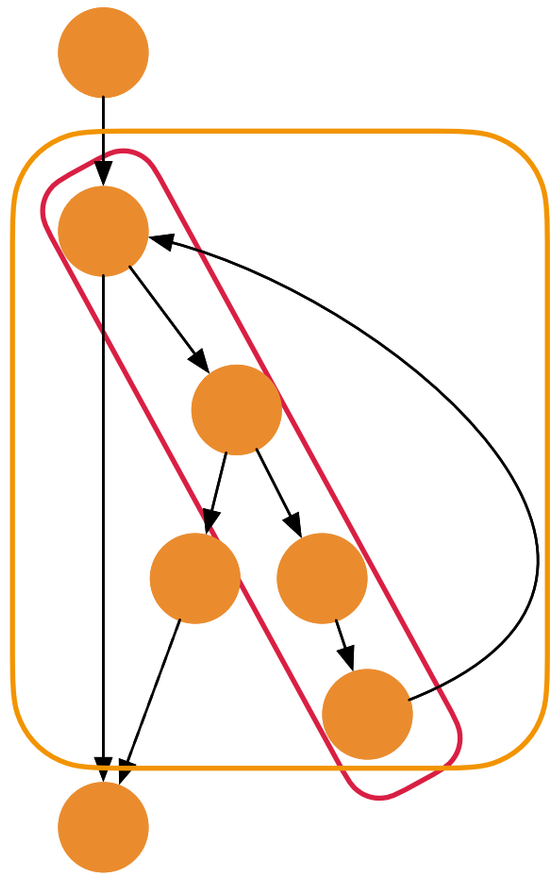
- Requires semantics preservation patterns!

- Multiple entries: choose entry with highest in-degree, virtualize other incoming edges
- If not while or do-while it's natural loop, choose any successor, which determines lexically contained loop nodes. Other exit edges are virtualized.
- Last resort: remove edges that prevent collapsing, preferably where either source does not dominate target or vice versa.

Determining Loop Nodes

- Lexically contained loop nodes?
- While loop with virtualization
- Lexically contained until successor (post-dominator?)!

```c
int f(void) {
  int a = 42;
  int b = 0;
  while (a) {
    if (b) {
      puts("c");
      break ;
    } else {
      puts("d");
    }
    a--;
    b++;
  }
  puts("e");
  return 0;
}
```



Example Decompilation

```
1   int f(void) {
2     int a = 42;
3     int b = 0;
4     while (a) {
5       if (b) {
6         puts("c");
7         break;
8       } else {
9         puts("d");
10      }
11      a--;
12      b++;
13    }
14    puts("e");
15    return 0;
16  }
```

(a) Original source code

```
1   t_reg32 f (void) {
2     t_reg32 var_20 = 42;
3     t_reg32 var_24;
4     for (var_24 = 0; var_20 != 0;
5     var_24 = var_24 + 1) {
6       if (var_24 != 0) {
7         puts("c");
8         break;
9       }
10      puts("d");
11      var_20 = var_20 - 1;
12    }
13    puts("e");
14    return 0;
15  }
```

(b) Phoenix decompiled output of (a) *with* new loop membership definition

References

- C. Cifuentes, K. J. Gough: Decompilation of binary programs. Software: Practice and Experience 25 (7), July 1995. Wiley
- Balakrishnan, G., Reps, T. (2004). Analyzing Memory Accesses in x86 Executables. In: Compiler Construction. CC 2004. LNCS, vol 2985. Springer
- G. Balakrishnan and T. Reps: WYSINWYX: What you see is not what you eXecute. ACM TOPLAS 32, 6, Article 23 (August 2010).
- E. Schwartz et al.: Native x86 Decompilation using Semantics-Preserving Structural Analysis and Iterative Control-Flow Structuring, USENIX Security, 2013
- J. Kinder: Reverse Engineering, Lecture at UBW, 2020